CmpE 260: Interpreter Project

Melang Language Design Report

Mert Seyhan, 2023400213 Metehan Seyhan, 2023400111

May 2026

1 Introduction

Melang is a small interpreted programming language that we designed and built from scratch for this project. The name comes from combining our first names (M*ert* and M*etehan*) with *lang*.

When building the interpreter, we followed the standard language processing pipeline. We wrote a custom character-by-character lexer to produce tokens, implemented a recursive descent parser to build the Abstract Syntax Tree (AST), and finally created a tree-walking evaluator that executes the program while maintaining an explicit environment.

In its current state, our language supports integers, booleans, strings, and lists. We also successfully implemented first-class functions with closures, recursive function definitions, and the ability to switch between static and dynamic scoping at runtime.

2 Scoping Decision

We implemented **static (lexical) scoping** as the default behavior for Melang. However, we also wanted to explore how dynamic scoping works, so we made it available via the `--scope dynamic` command-line flag (Bonus B4).

Under static scoping, our interpreter resolves free variables based on the environment where the function was *defined*. Under dynamic scoping, it looks at the environment where the function is currently being *called*.

Why we chose static scoping

We realized that static scoping makes programs much easier to read and reason about. When looking at a function definition in the source code, we can easily determine exactly which variables it can access without needing to know the entire execution path. On the other hand, dynamic scoping requires us to track the call stack at runtime to understand where a variable comes from, which makes debugging quite difficult.

This is why most practical languages we use (like Python, Java, and C) default to static scoping. Dynamic scoping is mostly found in older languages or shell scripting where implicit argument passing might be more convenient.

Concrete example where the two strategies differ

We used the following test case to verify our scoping implementations:

```
let x = 1;

let f = fun() -> x end;

let call_with_x = fun(newx) ->
```

```
  let x = newx;
  f()
end;

print(call_with_x(99));
```

- **Static scoping:** `f` was defined in the global scope where `x = 1`, so `f()` returns `1`. The output is `1`.
- **Dynamic scoping:** `f` is called inside the `call_with_x` function, where the local `x` is `99`. The output is `99`.

Running our interpreter with `--scope static` and `--scope dynamic` on this exact source code produces these different outputs successfully.

3 Evaluation Strategy

For Melang, we decided to use **call-by-value** (strict evaluation). Before our interpreter executes a function call, it fully evaluates all the argument expressions down to their final values. Only after that, it binds those values to the function's parameters in a newly created environment frame.

Call-by-value vs. call-by-name

If we had chosen **call-by-name**, the argument expressions would be passed into the function unevaluated. The interpreter would then re-evaluate the expression from scratch every single time the parameter is referenced inside the function body.

We saw that this difference becomes very important when an argument contains a side effect (like modifying a variable). Consider this scenario:

```
let counter = 0;

let increment = fun() ->
  counter = counter + 1;
  counter
end;

let use_twice = fun(v) ->
  v + v
end;

print(use_twice(increment()));
```

- **Call-by-value (Our implementation):** `increment()` is evaluated once before jumping into `use_twice`. `counter` becomes `1`, so `v` gets bound to `1`. Then `1 + 1` equals `2`. The output is `2`.
- **Call-by-name:** The `increment()` expression would be passed as `v`. Since `v` is used twice, the function increments the counter twice. First use returns `1`, second use returns `2`. Then `1 + 2` equals `3`. The output would be `3`.

By implementing call-by-value, we make sure our evaluator processes every argument node into a strict value before creating the new function frame.

4 Closure Representation

To handle first-class functions, we implemented closures. In our code, a closure is simply an instance of the `Closure` Python class, which stores three crucial pieces of information:

- **params**: A list of the parameter names as strings.

- **body**: The AST node representing the function's code block.
- **env**: A reference to the environment dictionary that was active at the exact moment the `fun` expression was evaluated.

When our interpreter evaluates something like `fun(x) -> x + 1 end`, it creates a `Closure` object and saves a pointer to the current environment. Later, when the user calls this function, we create a new scope frame. If we are in static scoping mode, the parent of this new frame is set to the saved environment. If we are in dynamic mode, the parent is set to the caller's environment.

The most important detail here is that `env` is a reference, not a deep copy. This means the closure and its surrounding outer scope share the exact same environment dictionary. This shared reference allows mutations to persist across function calls, which is necessary for cases like mutable counters:

```
let make_counter = fun(start) ->
  let count = start;
  fun() ->
    count = count + 1;
    count
  end
end;

let c = make_counter(0);
print(c());  (* Prints 1 *)
print(c());  (* Prints 2 *)
```

Because the inner function captures a reference to the frame containing `count`, each call to `c()` modifies the same variable.

5 Environment Management

We kept our environment model simple but effective. An environment in Melang is just a Python dictionary containing two keys: `bindings` (which maps string variable names to their values) and `parent` (a reference to the enclosing outer frame, or `None` if it is the global frame).

We defined three main operations to interact with environments:

- **extend**: Adds a new binding to the current frame's dictionary. We use this for `let` statements and for assigning arguments to parameters during function calls.
- **lookup**: Searches the current frame for a variable. If it's not there, it recursively checks the parent frame all the way up to the global frame. It raises a runtime error if the variable is undefined.
- **update**: Used for assignment operations (`x = expr`). It walks up the environment chain to find where the variable was originally defined and modifies it there. It does not create a new local variable.

In our language, only function calls create new environment frames. `if` blocks and `while` loops do not create new scopes, meaning a variable declared inside an `if` statement will exist in the surrounding function or global frame.

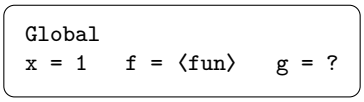
Required tracing exercise

Here is how we trace the following program step-by-step under static (lexical) scoping:

```
let x = 1;
let f = fun(a) ->
  let y = a + x;
  fun() ->
    x = x + 1;
    y + x
```

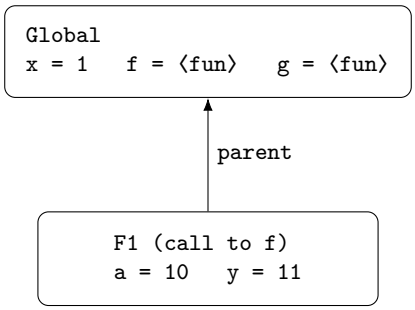
```
  end
end;
let g = f(10);
print(g());
print(g());
```

Step 1: Global frame after the first three let statements.



Step 2: Evaluating `f(10)`.

Our interpreter creates a new frame **F1** for the function call. Its parent is **Global** because `f` was defined there. We bind `a = 10` in **F1**. Then the statement `let y = a + x` looks up `x` from the Global frame, calculates `10 + 1 = 11`, and binds `y = 11` in **F1**. The inner `fun()` captures **F1** as its environment, creating a closure. This closure is returned and assigned to `g` in the Global frame.



Step 3: First call to `g()`.

A new frame **F2** is created. Its parent pointer is set to **F1** (because that is the environment captured by the closure).

- `x = x + 1`: The lookup operation for `x` checks **F2**, then **F1**, and finally finds it in **Global** as `1`. The `update` operation modifies it in **Global**. Now `x = 2` in **Global**.
- `y + x`: The lookup for `y` finds it in **F1** (`11`). The lookup for `x` finds it in **Global** (2). Result: $11+2 = 13$.

Output of first `print(g())`: **13**.

Step 4: Second call to `g()`.

A new frame **F3** is created, again with its parent pointing to **F1**.

- `x = x + 1`: `x` is found in **Global** where it is currently `2`. It is updated to `3`.
- `y + x`: `y` is found in **F1** (`11`, unchanged). `x` is found in **Global** (`3`). Result: $11 + 3 = 14$.

Output of second `print(g())`: **14**.

Why does `y` not change? `y` was bound in **F1** when `f(10)` was initially called. The inner assignment only updates `x`. Because `x` lives in **Global** and `y` lives in **F1**, modifying one has no effect on the other.

6 Challenges Faced

Recursive bindings. The most tricky part of the implementation for us was making recursive functions work, such as `let factorial = fun(n) -> ... factorial(n-1) ... end`. When the `fun` expression is being evaluated, our interpreter needs `factorial` to already exist in the environment so the closure can capture it. But at that exact moment, the evaluation hasn't finished yet. The solution we came up with is

to first bind the variable name to a dummy string (like `"DUMMY_PLACEHOLDER"`). Then we evaluate the `fun` expression (which safely captures the environment containing this placeholder). Finally, we overwrite that placeholder with the actual evaluated closure. Since closures hold references to environment dictionaries, this overwrite naturally becomes visible inside the recursive function.

Assignment vs. let semantics. Getting the distinction between `let x = expr` (creating a new binding) and `x = expr` (updating an existing binding) right took some debugging. If we accidentally created new local bindings during assignments, our closures wouldn't be able to mutate captured outer variables. We had to carefully separate our `add_var_to_env` and `update_var_in_env` logic to make sure the state persists correctly.

Runtime type checking for Booleans and Integers. We realized early on that in Python, `bool` is technically a subclass of `int`. This means if we used standard OOP checks like `isinstance(True, int)`, it would return `True`. To prevent our interpreter from silently treating boolean values as integers and allowing weird math operations, we completely avoided `isinstance()` and strictly used `type(val) == bool` and `type(val) == int` to enforce pure type constraints.

Integer division truncation direction. Python's `//` operator rounds towards negative infinity, but the project specification explicitly requires rounding towards zero (to match C and Java semantics). For example, evaluating $-7/2$ should result in -3, not -4. To solve this, we skipped the floor division operator entirely and simply used the `int(a / b)` conversion, which naturally truncates the decimal part towards zero in Python.

7 Grammar Ambiguity

We encountered two potential ambiguities while designing our grammar, and here is how we resolved them:

Assignment vs. equality. When the parser sees an identifier followed by an equals sign (`x = ...`), it could either be an assignment statement or the beginning of a comparison expression (`x == y`). We solved this right at the lexer level by making sure a single `=` and a double `==` generate two completely distinct token types (`ASSIGN` vs `EQ`). In our parser, if we see an identifier and the peeked token is `ASSIGN`, we immediately parse it as an assignment. If it is `EQ`, it falls through to our comparison parsing rules.

Dangling else. In many programming languages, a structure like `if A then if B then x else y` creates an ambiguity because it's unclear which `if` the `else` belongs to. We avoided this problem completely by requiring an explicit `end` keyword for every `if` block. Because our rule is strictly `if expr then block else block end`, the boundaries are mathematically clear, and the dangling else problem never occurs.